Google Cloud FinOps — Billing, Budgets & Cost Governance

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The outcome . . . 4

The approach — safe and governed by design . . . 4

Why it is safe . . . 4

The result . . . 4

Solution Overview 5

Google Cloud FinOps — Billing, Budgets & Cost Governance . . . 5

Requirements 7

1. Billing Account Administrator (or Costs Manager) access . . . 7

2. A dedicated project and BigQuery dataset for the billing export . . . 8

3. A standardized labeling convention with measurable coverage . . . 8

4. Recommender API enabled for optimization analysis . . . 9

5. A notification channel for budget alerts . . . 9

6. A named pilot scope and a FinOps operating owner . . . 10

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions 10

1. A Google Cloud Organization linked to Cloud Identity / Workspace . . . 10

2. Organization-level administrative access . . . 11

3. A Cloud Billing account . . . 11

4. The gcloud / IaC toolchain and API enablement . . . 11

5. A resource-hierarchy, naming & environment design . . . 12

6. Security, network & governance stakeholders and policy . . . 12

Implementation 12

Phase 1 — Assess (read-only) . . . 13

Phase 2 — Design . . . 13

Phase 3 — Configure . . . 14

Phase 4 — Pilot (`FinOps – Pilot – v1`) . . . 15

Phase 5 — Validate . . . 16

Phase 6 — Expand (org-wide) . . . 16

Rollback (org-wide changes) . . . 16

Testing & Validation 17

T1 — Billing account and access . . . 17

T2 — Detailed billing export is flowing . . . 17

T3 — Discovery baseline matches reality . . . 18

T4 — Attribution views resolve and cost rolls up . . . 18

T5 — Unattributed spend is measured and within target . . . 18

T6 — Pilot budget exists and alerts correctly . . . 18
T7 — Savings analysis is read-only and produces a backlog . . . 19
T8 — Guardrail check passes org-wide (post-Expand) . . . 19
T9 — Rollback is clean . . . 19
Troubleshooting 19
The billing export table never appears in BigQuery . . . 19
Attribution views error or totals do not reconcile . . . 20
Unattributed spend is high . . . 20
Budget alerts do not fire . . . 20
Budget provisioner: "PERMISSION_DENIED" or nothing happens . . . 21
Savings analyzer returns nothing or a permission error . . . 21
Guardrail check fails in CI but the console looks fine . . . 22
Authentication / ADC problems . . . 22
Best Practices 22
Make the BigQuery export the single source of truth . . . 22
Treat labeling as the foundation of attribution . . . 22
Design budgets to inform, not to block . . . 23
Keep the tooling safe-by-default . . . 23
Keep optimization a governed loop, not a one-off . . . 23
Run FinOps as an operating rhythm . . . 24
Respect the boundaries . . . 24
Deliverables Summary 24
Documentation . . . 24
Python tools (5) . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary

Cloud spend without FinOps becomes a surprise bill that nobody owns. As the enterprise adopts Google Cloud across many teams and projects, cost grows faster than the ability to see it, attribute it, or control it — and the first symptom is usually an over-budget invoice with no clear owner. This solution establishes **FinOps for Google Cloud**: a durable operating model that makes every dollar of spend **visible, attributable, and governed.**

The outcome

- **Visible.** Detailed billing export lands every line item in a BigQuery dataset — the

authoritative, queryable, durable record of spend, replacing short-lived console screenshots.

- **Attributable.** Cost rolls up along the folder/project hierarchy and along standardized labels

(`cost-center`, `team`, `environment`, `app`), so every charge reaches an accountable owner and "unattributed spend" becomes a tracked, shrinking backlog.

- **Governed.** Cloud Billing budgets with threshold and forecast alerts warn owners *before* the

month closes; rightsizing and committed-use-discount opportunities are surfaced as an owned savings backlog; and a recurring showback report keeps spend a normal part of how teams plan — extendable to Workspace and Gemini licensing.

The approach — safe and governed by design

The rollout follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback for every org-wide change. It is proven on a small **pilot scope** before any organization-wide budget or export change is applied.

The package includes five Python/`gcloud` automation tools that are **safe by default**: discovery, reporting, guardrail validation, and savings analysis are strictly **read-only**, and the only tool that writes — the budget provisioner — makes **no change unless `--apply` is passed** and prints its plan first. No service-account keys are created or committed and no secrets live in code; authentication uses Application Default Credentials with least-privilege roles.

Why it is safe

Budgets **alert but never cap** spend, and attribution views are derived, non-destructive, and re-creatable. The one genuinely irreversible FinOps action — purchasing a committed-use discount — is deliberately **kept out of the automation** and left to an explicit, human-approved decision by finance/FinOps. Financial, chargeback, and contractual determinations remain with the organization's finance and cloud-governance functions; this solution provides the technical method and the evidence.

The result

Google Cloud (and, by extension, Workspace/Gemini) spend made **visible, attributable, and controlled** — budgets alerting, savings identified, showback in place — delivered self-guided, tested, and at a fraction of the cost of a consulting engagement. No more surprise bills.

Solution Overview

Google Cloud FinOps — Billing, Budgets & Cost Governance

Cloud spend without FinOps becomes a surprise bill that nobody owns. Google Cloud makes it trivially easy to spin up resources across many projects and teams, and just as easy to lose sight of what each of them costs, who is accountable for it, and whether any of it is waste. This solution establishes **FinOps for Google Cloud** — a disciplined operating model that makes every dollar of spend **visible, attributable, and governed**.

FinOps here is not a one-time cleanup. It is a durable capability built on four load-bearing mechanisms, each of which this solution stands up and hands to the organization:

1. **Visibility — detailed billing export to BigQuery.** Google Cloud's console cost reports are

useful but aggregated and short-lived. The authoritative, queryable record of every line item is the **detailed usage cost export to BigQuery**. Once enabled, every SKU, project, label, and credit lands in a dataset the organization can query, join, and report on indefinitely.

1. **Attribution — cost broken down by the resource hierarchy and labels.** Spend is only

controllable when someone owns it. This solution attributes cost along the **folder/project hierarchy** (org → folder → project) and along **labels** (`cost-center`, `team`, `environment`, `app`) so that every charge rolls up to an accountable owner — a team, a cost center, or an application.

1. **Control — budgets and alerts.** Cloud Billing **budgets** with **threshold alert rules** turn

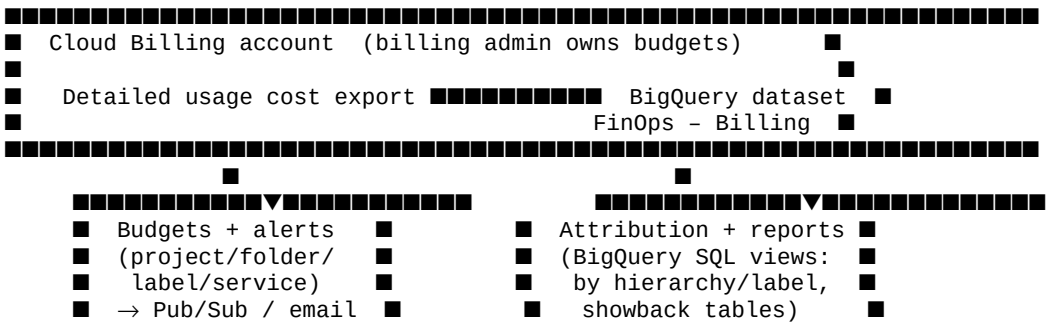
spend from something discovered after the fact into something that pages an owner *before* the month closes. Budgets scoped to projects, folders, labels, or services give each owner a guardrail on their own spend.

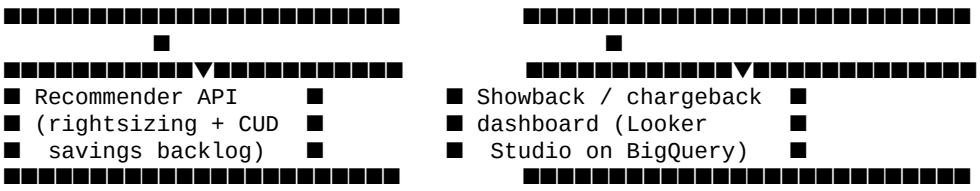
1. **Optimization — committed-use discounts (CUD) and rightsizing.** Google Cloud surfaces

rightsizing recommendations (idle/oversized VMs, unattached disks) and **committed-use discount** opportunities through the Recommender API. This solution turns those recommendations into a reviewed, owned savings backlog rather than an ignored console tab.

On top of these, the solution establishes a **showback/chargeback** cadence — a recurring report that shows each team what it spent, so cost becomes a normal part of how the organization plans and operates. The same model **extends to Google Workspace and Gemini licensing**: once the enterprise is attributing GCP spend by owner, the same showback discipline covers subscription/licensing cost.

The reference design





Everything is anchored on the **billing account** and the `FinOps – Billing Export – Prod` BigQuery dataset. Budgets, attribution views, savings analysis, and dashboards all read from that single authoritative export.

How it works, end to end

- **Billing export** is enabled on the billing account, writing the **detailed usage cost** export

(and, optionally, the **pricing** export) into a dedicated BigQuery dataset. This is the source of truth for all downstream reporting.

- **Attribution views** are BigQuery SQL views built on the export table that group cost by

`project.id`, by folder (via the resource ancestry), and by the label keys the organization standardized on. Unlabeled/unattributed spend is itself a first-class metric — it is the backlog of things that need an owner.

- **Budgets** are created against the billing account and scoped (all-projects, a folder's projects,

a label, or specific services) with **alert threshold rules** (e.g. 50/90/100% of a monthly amount, plus a forecasted-spend rule) that notify by email and/or a **Pub/Sub topic** for automation.

- **Optimization** reads the **Recommender API** (`google.compute.instance.MachineTypeRecommender`,

idle-resource recommenders, and commitment/CUD recommenders) to produce a savings backlog with an owner and an estimated monthly saving per item.

- **Showback** is a scheduled report (BigQuery query → table/dashboard) delivered to each team on a

cadence, closing the loop so spend stays owned.

Deployment model

This solution follows the house model used across every Google Cloud solution: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback for any org-wide change.

- **Assess** current billing configuration, hierarchy, labeling coverage, and existing budgets

(read-only — the discovery tool does exactly this).

- **Design** the attribution model (which labels, which hierarchy roll-ups), the budget policy

(who gets which budget at which thresholds), and the showback cadence.

- **Configure** the billing export, attribution views, budgets, and dashboards.

- **Pilot** on a single **pilot scope** (`FinOps – Pilot – v1`: one folder or a small set of

projects) before touching the whole organization.

- **Validate** against the gates: spend is attributable, budgets alert correctly, savings are

identified.

- **Expand** to the full organization once the pilot passes, keeping the rollback documented.

Benefits

- **No more surprise bills** — budgets alert owners before, not after, the month closes.
- **Every dollar has an owner** — cost rolls up to a team/cost-center via hierarchy and labels.
- **Waste becomes a backlog** — rightsizing and CUD opportunities are surfaced, owned, and tracked.
- **Durable, queryable history** — the BigQuery export outlives the console's short retention.
- **Extensible** — the same showback discipline covers Workspace and Gemini licensing spend.

Limitations and boundaries

- **This solution provides the technical FinOps method and evidence; it does not make financial or contractual decisions.** Chargeback policy, cost-center mappings, and commitment-purchase approvals are business decisions owned by finance and the cloud-governance/FinOps function.
- **CUD purchases move money and are commitments** — the tooling *identifies* opportunities (read-only); the actual purchase is an explicit, approved, human action, never automated here.
- **Billing export is not retroactive.** Enabling the export begins capturing data from that point forward; historical detail before enablement is limited to what the console retained.
- **Preview/version-dependent surfaces are flagged.** Where a Recommender, budget field, or export schema element is version- or preview-dependent, verify it against the current Google Cloud documentation for your organization rather than assuming the behavior described here.
- **Labeling is a prerequisite discipline, not a magic switch.** Attribution quality is bounded by labeling coverage; the solution measures and drives that coverage but cannot invent an owner for a resource nobody labeled.

Requirements

This solution assumes the **shared Google Cloud Platform foundation** described in `global-requirements-gcp.md` — a GCP Organization linked to Cloud Identity/Workspace, organization-level administrative access, a Cloud Billing account, the `gcloud`/Python toolchain with API enablement, a resource-hierarchy/naming/labeling design with a named pilot scope, and engaged security/network/governance/FinOps stakeholders. Verify those first.

The prerequisites below are the ones **specific to FinOps, billing export, and cost governance**. Each lists what it is, why it is required, how to verify it, and how to implement it if missing.

1. Billing Account Administrator (or Costs Manager) access

- **Description:** An identity holding `roles/billing.admin` on the target Cloud Billing account (or, for scoped work, `roles/billing.costsManager` to manage budgets and `roles/billing.viewer` to read cost). Enabling the export and creating budgets operate at the

billing-account level, not the project level.

- **Why required:** Billing export configuration and budget creation are billing-account resources.

Project-level Owner/Editor is not sufficient; the operator must be authorized on the billing account.

- **How to verify:** `gcloud billing accounts get-iam-policy BILLING_ACCOUNT_ID` and confirm the

operator's principal holds `roles/billing.admin` (or `costsManager`). `gcloud billing accounts list` must show the account as `OPEN`.

- **How to implement if missing:** A billing-account admin grants the role under change control:

`gcloud billing accounts add-iam-policy-binding BILLING_ACCOUNT_ID --member=user:EMAIL --role=roles/billing.admin`. Keep this access audited and MFA-protected.

2. A dedicated project and BigQuery dataset for the billing export

- **Description:** A dedicated GCP project (e.g. `finops-billing-prod`) hosting the BigQuery dataset

`FinOps – Billing Export – Prod` that will receive the detailed usage cost export, with the **BigQuery API** (`bigquery.googleapis.com`) enabled and the project **linked to a billing account**.

- **Why required:** The detailed usage cost export writes to a BigQuery dataset; a dedicated project

isolates the FinOps data, its IAM, and its own (small) query cost from workload projects.

- **How to verify:** `gcloud services list --enabled --project FINOPS_PROJECT` shows

`bigquery.googleapis.com`; `bq ls --project_id FINOPS_PROJECT` shows the dataset (once created).

- **How to implement if missing:** Create the project, link billing, enable the API

(`gcloud services enable bigquery.googleapis.com --project FINOPS_PROJECT`), and create the dataset (`bq --location=US mk --dataset FINOPS_PROJECT:finops_billing_export`) in the region the org's data policy allows. Choose the dataset location deliberately — it cannot be changed after export begins.

3. A standardized labeling convention with measurable coverage

- **Description:** An agreed set of **billing/resource labels** — at minimum `cost-center`, `team`,

`environment`, and `app` — applied to projects and (where supported) resources, plus a target labeling-coverage threshold (e.g. ">= 95% of spend attributable").

- **Why required:** Attribution and showback are only as good as labeling coverage. Cost that carries

no `team`/`cost-center` label cannot be rolled up to an owner and becomes "unattributed" spend.

- **How to verify:** Query the billing export for the share of cost whose `labels`/`project.labels`

contain the required keys (the report generator tool reports exactly this "unattributed %").

- **How to implement if missing:** Define the label keys and allowed values, apply project-level

labels (`gcloud projects update PROJECT_ID --update-labels team=...,cost-center=...`), and set labels on billable resources at creation. Consider an org policy / Terraform standard so new resources are labeled by default. Note: **billing export reflects labels going forward** — relabeling does not rewrite historical rows.

4. Recommender API enabled for optimization analysis

- **Description:** The **Recommender API** (`recommender.googleapis.com`) enabled on the projects to be

analyzed, so the tooling can read **rightsizing** (machine-type), **idle-resource**, and **committed-use-discount (CUD)** recommendations. Read access requires `roles/recommender.viewer` (or the specific recommender viewer roles).

- **Why required:** The savings analysis reads recommendations from this API. Without it enabled and

readable, the CUD/rightsizing analyzer has no source data.

- **How to verify:** `gcloud services list --enabled --project PROJECT` shows

`recommender.googleapis.com`; `gcloud recommender recommendations list --project=PROJECT --location=REGION --recommender=google.compute.instance.MachineTypeRecommender` returns without a permission error.

- **How to implement if missing:** `gcloud services enable recommender.googleapis.com --project PROJECT`

and grant `roles/recommender.viewer`. Note that recommender availability and specific recommender IDs are **version-dependent** — confirm the current recommender IDs for CUD and rightsizing in the Google Cloud Recommender documentation for your organization.

5. A notification channel for budget alerts

- **Description:** A destination for budget alerts — **billing-account admins/users by email** and/or

a dedicated **Pub/Sub topic** (`FinOps – Budget Alerts – Prod`) for programmatic handling (routing to chat, ticketing, or automation). Cloud Monitoring notification channels may also be used for downstream alerting.

- **Why required:** A budget with no notification target is invisible. Alerts must reach an accountable

owner (email) and, for automation, a machine-readable channel (Pub/Sub).

- **How to verify:** Confirm the email recipients on the budget's alert rule, and (if used)

`gcloud pubsub topics describe FinOps-Budget-Alerts-Prod` succeeds with the billing service account granted `roles/pubsub.publisher` on the topic.

- **How to implement if missing:** Create the topic

(`gcloud pubsub topics create FinOps-Budget-Alerts-Prod`), grant the Cloud Billing budget service agent publish rights, and reference it in the budget's `notificationsRule`. Email recipients are set by billing role or explicitly on the budget.

6. A named pilot scope and a FinOps operating owner

• **Description:** A designated **pilot scope** (`FinOps – Pilot – v1`: one folder or a small set of projects) to validate the model on before org-wide rollout, and a named **FinOps owner** (a person or the cloud-governance/FinOps function) who owns the budget policy, the showback cadence, and the savings backlog.

• **Why required:** The deployment model is pilot-first; org-wide budgets/alerts must not be the first test. FinOps is an ongoing practice, so it needs a standing owner — not just a one-time setup.

• **How to verify:** Confirm the pilot folder/projects and the named owner with the platform and finance stakeholders.

• **How to implement if missing:** Nominate the pilot scope and the FinOps owner during Assess/Design, before any budget or export change is applied org-wide.

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions

These requirements apply to the **Google Cloud Platform (GCP)** infrastructure and zero-trust solutions in the **Google** category (landing zone, Cloud IAM, networking / VPC Service Controls, Security Command Center, FinOps, and BeyondCorp Enterprise). They are the common foundation each GCP solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every GCP solution).*** *The* ***Google Cloud Console*** *(console.cloud.google.com), the* `gcloud` ***CLI****, and* ***Cloud Identity / Google Admin*** *are authored at exact, current click-by-click / command detail from Google's public documentation. Where a capability, API, or default is* ***version- or preview-dependent*** *(new SCC tiers, BeyondCorp features, org-policy constraints, service-perimeter behaviors), it is* ***cited or flagged "verify in your organization"*** *— never invented.* ***Infrastructure-as-Code*** *(Terraform, the Cloud Foundation Toolkit / Terraform blueprints, Config Controller) is used at the* ***pattern level*** *— the solutions give the design, the* `gcloud`*/console procedure, and representative IaC, and defer provider/module internals to the tool's own docs. Automation is delivered as* ***Python*** *(Cloud Client Libraries /* `gcloud` *wrappers), never with hardcoded credentials or keys.*

1. A Google Cloud Organization linked to Cloud Identity / Workspace

• **Description:** A **GCP Organization** resource (the root of the resource hierarchy), which exists once the domain's **Cloud Identity** or **Google Workspace** account is set up. Solutions build the folder/project hierarchy beneath it.

• **Why required:** Org-level policy, IAM, VPC Service Controls, and SCC all operate at or below the Organization node; without it there is no place to anchor centralized governance.

• **How to verify:** In the Cloud Console resource selector (or `gcloud organizations list`), confirm

an Organization resource exists and its ID.

- **How to implement if missing:** Establish Cloud Identity/Workspace for the domain so the Organization is created, then verify domain ownership.

2. Organization-level administrative access

- **Description:** An account (ideally a **break-glass super-admin** plus a delegated **Organization Administrator** / least-privilege roles the solution names) able to set org policies, create folders/projects, and grant IAM at the Organization node.
- **Why required:** Foundational changes (resource hierarchy, org policies, IAM bindings, VPC-SC perimeters) require Organization-level authority.
- **How to verify:** Confirm the account holds `roles/resourcemanager.organizationAdmin` (and the specific roles each solution lists) via `gcloud organizations get-iam-policy`.
- **How to implement if missing:** Have a super-admin grant the roles under change control; keep break-glass accounts audited and MFA-protected.

3. A Cloud Billing account

- **Description:** An active **Cloud Billing account** linked (or linkable) to the projects the solution provisions, with a billing administrator available.
- **Why required:** Projects that host resources (networks, SCC, workloads) must be linked to billing; FinOps/budget work operates directly on the billing account.
- **How to verify:** `gcloud billing accounts list` shows an OPEN account; confirm billing-admin access.
- **How to implement if missing:** Create/associate a billing account (Google or reseller) before provisioning billable resources.

4. The gcloud / IaC toolchain and API enablement

- **Description:** A workstation or pipeline with the **Google Cloud SDK (`gcloud`, `gsutil`)**, **Python 3.10+** with the **Cloud Client Libraries**, and (where used) **Terraform**; plus the ability to **enable the required Google Cloud APIs** on target projects.
- **Why required:** Configuration, automation, and the provided Python tools run through these; most services require their API to be enabled first.
- **How to verify:** `gcloud version`, `python --version`, and `gcloud services list --enabled` on a target project.

• **How to implement if missing:** Install the SDK + libraries; enable the named APIs (`gcloud services enable …`) before running the steps.

5. A resource-hierarchy, naming & environment design

• **Description:** An agreed **folder/project structure** (e.g. by department/environment), a **naming/labeling convention**, and the **environments** in scope (prod / non-prod / shared), plus a **pilot project or folder** to validate on before org-wide rollout.

• **Why required:** IAM, org policy, networking, and cost allocation all key off the hierarchy and labels; every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model.

• **How to verify:** Confirm the intended hierarchy, naming convention, and a named pilot scope with the platform owners.

• **How to implement if missing:** Define the hierarchy and naming standard (this is often the first solution — the Landing Zone) before building dependent controls.

6. Security, network & governance stakeholders and policy

• **Description:** Engagement with the **security (CISO/SecOps), network, and cloud-governance / FinOps** stakeholders whose policies the configuration must satisfy (least-privilege model, allowed regions, data-residency, network egress, threat-response ownership, budget/showback rules).

• **Why required:** Org policies, IAM boundaries, perimeter design, and cost governance must align to enterprise security and financial policy, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named approvers/owners for security, networking, and cost.

• **How to implement if missing:** Identify the owners and capture the policy requirements before applying org-wide guardrails.

*The deployment model across every GCP solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for org-wide changes. Automation is delivered as **Python** (`gcloud` / Cloud Client Libraries) — never with hardcoded credentials, exported service-account keys committed to disk, or secrets in code.*

Implementation

Follow the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback for every org-wide change. Every artifact is named with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in testing, troubleshooting, and the scripts.

Artifacts created in this solution (names are reused verbatim later):

Artifact	Name
FinOps host project	`finops-billing-prod`
Billing export dataset	`FinOps – Billing Export – Prod` (BigQuery dataset id `finops_billing_export`)
Attribution views	`FinOps – Attribution Views – v1`
Pilot budget	`FinOps – Budget – Pilot – v1`
Org-wide budgets	`FinOps – Budget – Org – v1`
Budget alert topic	`FinOps – Budget Alerts – Prod` (Pub/Sub `FinOps-Budget-Alerts-Prod`)
Savings backlog	`FinOps – Savings Backlog – v1`
Showback dashboard	`FinOps – Showback – v1`
Pilot scope	`FinOps – Pilot – v1`

Set once and reuse in every command below: `BILLING=<BILLING_ACCOUNT_ID>` · `FINOPS_PROJECT=finops-billing-prod` · `DATASET=finops_billing_export`.

Phase 1 — Assess (read-only)

1. **Inventory the billing baseline.** Run the discovery tool to capture the current state without changing anything:

```
python finops-discovery.py --billing-account "$BILLING" --json baseline.json
```

It reports: billing account status and linked projects, whether a detailed export is already configured, existing budgets and their thresholds, the resource hierarchy (org → folders → projects), and the labeling keys present on projects. This is the **Assess evidence**.

1. **Review labeling coverage and hierarchy.** Confirm the required label keys (`cost-center`, `team`, `environment`, `app`) exist on projects and note the gaps. Confirm the folder structure the org will attribute along.

1. **Record what already exists.** Note any pre-existing export or budgets so Configure is additive and idempotent — never delete an existing budget you did not create.

Assess gate: you have a written baseline (billing status, export state, existing budgets, hierarchy, labeling coverage). Do not proceed until the baseline is captured.

Phase 2 — Design

1. **Design the attribution model.** Decide the roll-ups: by `project.id`, by folder (resource ancestry), and by each label key. Decide how "unattributed" is defined (missing `team` or `cost-center`). Record the target coverage (e.g. >= 95% of spend attributable).

1. **Design the budget policy.** For the pilot and then the org: which scope gets a budget (all

projects, a folder, a label, a service), the monthly amount or a last-period-based amount, and the alert thresholds (e.g. **50%, 90%, 100% of actual**, plus a **forecasted-spend** rule). Decide notification: email recipients and/or the `FinOps – Budget Alerts – Prod` Pub/Sub topic.

1. **Design the showback cadence.** Who receives the showback report, at what interval (e.g. monthly),

and in what form (BigQuery table + Looker Studio dashboard `FinOps – Showback – v1`).

Design gate: attribution model, budget policy, and showback cadence are written down and approved by the FinOps owner and finance stakeholder.

Phase 3 — Configure

3a. Enable detailed billing export to BigQuery

1. **Create the FinOps project and dataset** (if not already present from Requirements):

```
gcloud projects create "$FINOPS_PROJECT"
gcloud billing projects link "$FINOPS_PROJECT" --billing-account "$BILLING"
gcloud services enable bigquery.googleapis.com --project "$FINOPS_PROJECT"
bq --location=US mk --dataset "$FINOPS_PROJECT:$DATASET"
```

Choose the dataset location per the org's data-residency policy — **it cannot be changed later.**

1. **Enable the detailed usage cost export.** In the Cloud Console go to **Billing → Billing export

→ BigQuery export**, and under** Detailed usage cost **click** Edit settings**, select the** `finops-billing-prod` **project and the** `finops_billing_export` **dataset, and** Save**. (Enabling billing export is a console/API action on the billing account; confirm the exact panel labels in the current Cloud Billing documentation for your organization — the export type names are version-stable but the panel layout evolves.)

- Optionally also enable **Pricing** export into the same dataset for unit-price analysis.
- Export is **not retroactive** — data accrues from enablement forward. The

`gcp_billing_export_resource_v1_<BILLING_ACCOUNT_ID>` table appears within a few hours and populates over the next day.

1. **Confirm the export table appears:**

```
bq ls "$FINOPS_PROJECT:$DATASET"
```

Wait until a `gcp_billing_export_*` table exists and has rows before building views.

3b. Build attribution views

1. **Create the attribution views** (`FinOps – Attribution Views – v1`). Use the report generator to

emit the SQL and, with `--apply`, create the views in the dataset:

```
python finops-report.py --project "$FINOPS_PROJECT" --dataset "$DATASET" \
    --emit-views --out views.sql           # dry-run: writes SQL, creates nothing
python finops-report.py --project "$FINOPS_PROJECT" --dataset "$DATASET" \
    --emit-views --apply                    # creates the views
```

The views include: **cost-by-project**, **cost-by-label** (per required key), **cost-by-service**, and **unattributed-spend** (rows missing `team`/`cost-center`). Views read the export table so they always reflect the latest data.

3c. Provision budgets and alerts (pilot first — see Phase 4)

1. **Create the budget alert topic** (`FinOps – Budget Alerts – Prod`) if using Pub/Sub:

```
gcloud pubsub topics create FinOps-Budget-Alerts-Prod --project "$FINOPS_PROJECT"
```

Grant the Cloud Billing budget service agent `roles/pubsub.publisher` on the topic (see the Cloud Billing budgets-notifications documentation for the exact service-agent principal).

1. **Provision the pilot budget with the provisioner (dry-run by default):**

```
python finops-budget-provisioner.py --billing-account "$BILLING" \
    --name "FinOps – Budget – Pilot – v1" \
    --scope-projects PILOT_PROJECT_1,PILOT_PROJECT_2 \
    --amount 5000 --currency USD \
    --thresholds 0.5,0.9,1.0 --forecast-threshold 1.0 \
    --pubsub-topic FinOps-Budget-Alerts-Prod --dry-run       # prints the plan, creates nothing
```

Review the printed plan, then re-run with `--apply` to create the budget.

3d. Stand up the showback dashboard

1. **Build the showback dashboard** (`FinOps – Showback – v1`). Point **Looker Studio** at the

`finops_billing_export` dataset (or a showback summary table produced by `finops-report.py`), with one page per audience (per team / per cost-center) and a trend + top-services breakdown. (Dashboard construction is a Looker Studio task; the tool provides the underlying query/table.)

Phase 4 — Pilot (`FinOps – Pilot – v1`)

1. **Confirm the pilot is scoped, not org-wide.** The pilot budget `FinOps – Budget – Pilot – v1`

covers only `FinOps – Pilot – v1` (one folder or a few projects). No org-wide budget exists yet.

1. **Run the guardrail check** against the pilot to confirm the FinOps configuration is sound:

```
python finops-guardrail-check.py --billing-account "$BILLING" \
    --project "$FINOPS_PROJECT" --dataset "$DATASET" --json pilot-guardrail.json
```

It verifies: export is flowing (recent rows), the pilot budget exists with the expected thresholds and a live notification target, attribution views resolve, and unattributed spend for the pilot scope is below the target. All must pass.

1. **Trigger a proof alert (optional, safe).** Set a deliberately low pilot budget amount so the 50%

threshold fires, confirm the alert reaches email/Pub/Sub, then restore the real amount with the provisioner `--apply`. This proves the notification path end to end.

1. **Identify pilot savings:**

```
python finops-savings-analyzer.py --projects PILOT_PROJECT_1,PILOT_PROJECT_2 \
    --regions REGION --json pilot-savings.json
```

Read-only. Produces the `FinOps – Savings Backlog – v1` for the pilot: rightsizing, idle resources, and CUD opportunities with an estimated monthly saving and an owner per item.

Pilot gate (all must pass):

- Billing export has recent rows for the pilot scope.
- Pilot spend is attributable (unattributed % <= target).
- The pilot budget alerts correctly to its notification target.
- A savings backlog was produced.

If any gate fails, **do not expand.** Fix and re-run the guardrail check.

Phase 5 — Validate

1. **Validate against the gates for real data.** Re-run `finops-guardrail-check.py` and confirm every

check passes for the pilot. Confirm the showback dashboard renders the pilot teams' spend and that the numbers reconcile to the console's cost report for the same period (within rounding/credit timing).

1. **Review the savings backlog with owners.** Each item in `FinOps – Savings Backlog – v1` gets an

owner and a decision (act / defer / reject). **CUD purchases are commitments and money movement — they are approved and executed by a human, never by the tooling.**

Validate gate: guardrail check fully passes, dashboard reconciles, savings backlog is owned.

Phase 6 — Expand (org-wide)

1. **Create the org-wide budgets** (`FinOps – Budget – Org – v1`) with the provisioner, scoped as the

budget policy defined (all-projects and/or per-folder/per-label budgets), dry-run first:

```
python finops-budget-provisioner.py --billing-account "$BILLING" \
    --name "FinOps – Budget – Org – v1" --scope-all-projects \
    --amount 250000 --currency USD --thresholds 0.5,0.9,1.0 \
    --forecast-threshold 1.0 --pubsub-topic FinOps-Budget-Alerts-Prod --dry-run
```

Review the plan; re-run with `--apply`.

1. **Expand attribution + showback org-wide.** The attribution views already cover the whole export;

extend the showback dashboard to all teams/cost-centers and set the recurring showback cadence.

1. **Run the org-wide guardrail check** and confirm unattributed spend is below target across the

organization. Where it is not, drive labeling remediation (the report tool lists the biggest unattributed projects/services).

1. **Establish the operating rhythm.** Schedule: monthly showback delivery, monthly savings-backlog

review, and budget-alert triage as alerts fire. Assign these to the named FinOps owner.

Rollback (org-wide changes)

Every org-wide change is reversible:

- **Budgets:** delete a budget you created with

`python finops-budget-provisioner.py --delete --name "FinOps – Budget – Org – v1" --billing-account "$BILLING" --apply` (or `gcloud billing budgets delete`). Budgets are non-destructive — deleting one only stops alerts; it changes no spend and no resource. Never delete a budget you did not create.

- **Attribution views:** `bq rm -f "$FINOPS_PROJECT:$DATASET.<view>"` removes a view; views are

derived and safely re-creatable from `finops-report.py --emit-views`.

- **Billing export:** turn the export off in **Billing** → **Billing export** to stop new writes. The

historical dataset is retained; disabling only stops forward capture.

- **Savings backlog / dashboards:** these are read-only artifacts — no rollback needed; discard the

report file.

Because budgets and views are non-destructive and the analysis tools are read-only, the only truly irreversible FinOps action — **purchasing a committed-use discount** — is deliberately kept out of the tooling and left to an explicit, approved human decision.

Testing & Validation

Each procedure states what to run, the expected result, and the explicit **PASS/FAIL** signal. Run the pilot tests (T1–T6) before expanding org-wide; re-run T2–T8 after Expand. The artifact names are the same ones created in Implementation.

T1 — Billing account and access

- **Run:** `gcloud billing accounts describe "$BILLING"` and

`gcloud billing accounts get-iam-policy "$BILLING"`.

- **Expected:** the account shows `open: true`; the operator principal holds `roles/billing.admin`

(or `costsManager` + `viewer`).

- **PASS:** account OPEN and operator authorized. **FAIL:** account closed or operator lacks billing

role — resolve access before any other test.

T2 — Detailed billing export is flowing

- **Run:** `bq ls "$FINOPS_PROJECT:$DATASET"` then a freshness query:

```
bq query --use_legacy_sql=false \
'SELECT MAX(export_time) AS last_export, COUNT(*) AS rows_24h
 FROM `finops-billing-prod.finops_billing_export.gcp_billing_export_resource_v1_*`
 WHERE export_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 24 HOUR)'
```

- **Expected:** a `gcp_billing_export_*` table exists and `rows_24h > 0` with a recent `last_export`.

- **PASS:** recent rows present. **FAIL:** no table or no recent rows — export is not enabled or is

still initializing (allow up to ~24h after first enablement); see Troubleshooting.

T3 — Discovery baseline matches reality

- **Run:** `python finops-discovery.py --billing-account "$BILLING" --json baseline.json`.
- **Expected:** the tool reports the billing status, the export as configured, existing budgets, the hierarchy, and label keys — matching what the console shows.
- **PASS:** report reconciles with the console (billing export = configured, budgets listed). **FAIL:** discrepancies — investigate before trusting downstream tools.

T4 — Attribution views resolve and cost rolls up

- **Run:** query the cost-by-project and cost-by-label views for the last full month, e.g.

```
bq query --use_legacy_sql=false \
'SELECT * FROM `finops-billing-prod.finops_billing_export.v_cost_by_project`
 ORDER BY cost DESC LIMIT 20'
```

- **Expected:** cost aggregates per project and per label key; the total reconciles (within credit/rounding timing) to the console cost report for the same period.
- **PASS:** views return data and totals reconcile. **FAIL:** views error or totals diverge materially — re-emit views with `finops-report.py --emit-views --apply` and recheck the export table name.

T5 — Unattributed spend is measured and within target

- **Run:** `python finops-report.py --project "$FINOPS_PROJECT" --dataset "$DATASET" --unattributed`.
- **Expected:** a single "unattributed %" figure plus the top unattributed projects/services.
- **PASS:** unattributed % <= the target set in Design (e.g. <= 5%). **FAIL:** above target — drive labeling remediation on the listed projects, then re-run.

T6 — Pilot budget exists and alerts correctly

- **Run:** `python finops-guardrail-check.py --billing-account "$BILLING" --project "$FINOPS_PROJECT" --dataset "$DATASET" --json pilot-guardrail.json, then perform the safe proof-alert (temporarily lower FinOps – Budget – Pilot – v1 so the 50% threshold fires; confirm email/Pub/Sub receipt; restore the amount with --apply`).
- **Expected:** the guardrail check reports the pilot budget present with the expected thresholds and a live notification target; the proof alert is received.
- **PASS:** budget present, thresholds correct, alert delivered, amount restored. **FAIL:** budget missing, wrong thresholds, or no alert delivered — see Troubleshooting (notification path).

T7 — Savings analysis is read-only and produces a backlog

• **Run:** `python finops-savings-analyzer.py --projects PILOT_PROJECT_1 --regions REGION --json pilot-savings.json`.

• **Expected:** a backlog of rightsizing/idle/CUD recommendations with estimated monthly savings and an owner per item; the tool makes **no changes** (read-only).

• **PASS:** backlog produced, nothing modified. **FAIL:** permission error (grant `roles/recommender.viewer` / enable `recommender.googleapis.com`) or the tool attempts a write (it must not).

T8 — Guardrail check passes org-wide (post-Expand)

• **Run:** `python finops-guardrail-check.py --billing-account "$BILLING" --project "$FINOPS_PROJECT" --dataset "$DATASET" --scope org --json org-guardrail.json`.

• **Expected:** export flowing, org budgets (`FinOps – Budget – Org – v1`) present with correct thresholds and notification, attribution views resolve, org-wide unattributed % <= target.

• **PASS:** every check green. **FAIL:** any check red — the org is not fully governed; remediate the failing check and re-run before declaring the rollout complete.

T9 — Rollback is clean

• **Run:** on a throwaway test budget, `python finops-budget-provisioner.py --delete --name "FinOps – Budget – Test – v1" --billing-account "$BILLING" --apply, then confirm it is gone with gcloud billing budgets list --billing-account "$BILLING"`.

• **Expected:** only the named budget is removed; no other budget, view, or resource is touched; no spend changes.

• **PASS:** targeted budget deleted, everything else intact. **FAIL:** wrong budget removed or a view disappeared — restore from `finops-report.py --emit-views` and review the provisioner invocation.

Troubleshooting

Common failure modes when standing up and running FinOps on Google Cloud, with the resolution for each. Artifact names match those created in Implementation.

The billing export table never appears in BigQuery

• **Symptoms:** `bq ls "$FINOPS_PROJECT:$DATASET"` shows no `gcp_billing_export_*` table hours after enabling export; T2 fails.

- **Causes & fixes:**

- **Export was not actually saved.** Re-open **Billing** → **Billing export** → **BigQuery export**,

confirm **Detailed usage cost** shows the `finops-billing-prod` project and `finops_billing_export` dataset, and **Save** again.

- **Not enough time.** The first export table can take **up to ~24 hours** to appear and populate.

Wait, then recheck. Export is **not retroactive** — no historical rows appear.

- **Dataset in an incompatible location / project not billing-linked.** Confirm the dataset exists

(`bq ls`) and the FinOps project is linked to a billing account.

- **Permissions.** Enabling export requires billing-admin on the billing account; confirm with

`gcloud billing accounts get-iam-policy "$BILLING"`.

Attribution views error or totals do not reconcile

- **Symptoms:** T4 fails — a view errors, or the view total diverges materially from the console cost

report.

- **Causes & fixes:**

- **Wrong export table name.** The views wildcard `gcp_billing_export_resource_v1_*`. If you enabled

only the **standard** (non-resource) export, the table is `gcp_billing_export_v1_*` — adjust the view SQL (edit `EXPORT_TBL` in `finops-report.py`) and re-run `--emit-views --apply`.

- **Credits/rounding/timing.** Console reports and the export can differ slightly by credit

application and export lag; reconcile on a **closed** invoice month, not the in-progress one.

- **Views not created.** Run `python finops-report.py --project "$FINOPS_PROJECT" --dataset "$DATASET"

--emit-views --apply and confirm with bq ls`.

Unattributed spend is high

- **Symptoms:** T5 reports unattributed % above target; showback has a large "unattributed" bucket.

- **Cause:** projects/resources lack the `team`/`cost-center` labels.

- **Fix:** query `v_unattributed_spend` (ORDER BY cost DESC) for the biggest offenders, apply the

labels (`gcloud projects update PROJECT_ID --update-labels team=...,cost-center=...`), and enforce labels at creation (org policy / Terraform default). Remember relabeling is **not retroactive** — the number improves from the next export forward, not historically.

Budget alerts do not fire

- **Symptoms:** T6 proof alert never arrives by email or Pub/Sub.

- **Causes & fixes:**

- **No notification target.** `finops-guardrail-check.py` reports "a budget has a notification target" as FAIL. Add email recipients (by billing role or explicit) and/or set the Pub/Sub topic on the budget (`finops-budget-provisioner.py ... --pubsub-topic FinOps-Budget-Alerts-Prod --apply`).

- **Pub/Sub publish permission missing.** The **Cloud Billing budget service agent** must hold `roles/pubsub.publisher` on `FinOps-Budget-Alerts-Prod`. Grant it (see the Cloud Billing budgets-notifications docs for the exact service-agent principal).

- **Threshold not crossed / forecast only.** Actual-spend alerts fire only when spend crosses the percent; for early warning add a **forecasted-spend** rule (`--forecast-threshold`).

- **Budgets alert, they do not cap.** If you expected spend to *stop*, that is by design — a budget never blocks spend.

Budget provisioner: "PERMISSION_DENIED" or nothing happens

- **Symptoms:** `finops-budget-provisioner.py --apply` errors with permission denied, or the dry-run printed a command but nothing was created.

- **Causes & fixes:**

- **Missing role.** Budget create/delete needs `roles/billing.admin` or `roles/billing.costsManager` on the billing account. Grant it and retry.

- **You did not pass `--apply`.** By design the tool is dry-run and prints the command it *would* run; add `--apply` to execute.

- **Notify project not set.** If using `--pubsub-topic`, also pass `--notify-project` so the topic path resolves.

Savings analyzer returns nothing or a permission error

- **Symptoms:** T7 — `finops-savings-analyzer.py` prints "(skip)" lines or an empty backlog.

- **Causes & fixes:**

- **Recommender API disabled.** `gcloud services enable recommender.googleapis.com --project PROJECT`.

- **Missing role.** Grant `roles/recommender.viewer` (or the specific recommender viewer role).

- **Wrong location.** Rightsizing/idle recommenders are **regional/zonal** — pass the actual `--regions` where the VMs run; a project with no matching resources legitimately returns nothing.

- **Recommender ID changed.** Recommender IDs are **version-dependent**; verify the current IDs in

the Google Cloud Recommender docs and pass them via `--recommenders`.

Guardrail check fails in CI but the console looks fine

- **Symptoms:** `finops-guardrail-check.py` exits non-zero though the console appears healthy.
- **Causes & fixes:**
- **Export freshness window.** The check counts rows in the last 48h; a delayed export can trip it.

Confirm with T2's freshness query; if export genuinely lags, treat it as a real export issue.

- **Views listed via an unavailable surface.** The check falls back to `bq ls`; ensure `bq` is on

PATH and the dataset is readable.

- **Unattributed above** `--max-unattributed`**.** This is a *real* governance failure, not a tooling

bug — remediate labeling and re-run.

Authentication / ADC problems

- **Symptoms:** any tool errors with default-credentials-not-found or 401/403.
- **Fix:** authenticate with `gcloud auth login` and, for the Cloud Client Libraries path,

`gcloud auth application-default login`. **Never** create or commit a service-account key to work around this — use ADC or an attached/workload-identity service account, and confirm the principal holds the least-privilege roles the Requirements list.

Best Practices

Durable practices for running FinOps on Google Cloud after the initial rollout. These keep the model accurate, keep spend owned, and keep the optimization loop turning.

Make the BigQuery export the single source of truth

- **Query the detailed export, not the console screenshots.** The console cost report is for quick

looks; every number that goes into showback, chargeback, or a savings decision should trace to a BigQuery query on `FinOps – Billing Export – Prod`.

- **Never change the export dataset's location or delete its table.** Location is fixed at creation

and the export is not retroactive — losing the table loses history you cannot rebuild.

- **Version your attribution SQL.** Keep the `FinOps – Attribution Views – v1` SQL in source control

so a view change is reviewed and reproducible (`finops-report.py --emit-views` regenerates it).

Treat labeling as the foundation of attribution

- **Attribution quality == labeling coverage.** Track "unattributed %" as a first-class KPI and drive

it down; an attribution model on top of poor labels is precision on top of noise.

• **Enforce labels at creation.** Prefer org policy / Terraform defaults so new projects and resources are labeled with `cost-center`, `team`, `environment`, and `app` from day one, rather than chasing labels after the fact.

• **Remember relabeling is not retroactive.** The export reflects labels going forward; fix the label now, but historical rows keep the label they were exported with.

Design budgets to inform, not to block

• **Budgets alert; they do not cap.** A Cloud Billing budget does not stop spend — it notifies. Use it as an early-warning system, and if hard enforcement is needed, wire the Pub/Sub alert to your own automation deliberately (and carefully — capping can break production).

• **Always include a forecasted-spend rule.** A forecast threshold warns you *before* actual spend crosses the line, which is the whole point of a budget.

• **Scope budgets to owners.** Per-folder, per-project, or per-label budgets put the alert in front of the team that can act, instead of a single org budget that pages everyone and no one.

Keep the tooling safe-by-default

• **Read-only and dry-run first.** Run discovery, reporting, savings analysis, and guardrail checks freely (they never change anything). The provisioner only writes with `--apply` — review the dry-run plan every time.

• **Use ADC / the environment for auth — never keys in code.** Authenticate with `gcloud auth application-default login` or a workload-identity/attached service account. Do not create, download, or commit service-account keys, and never put a billing account id or secret in a script.

• **Grant least privilege.** `roles/billing.viewer` + `roles/recommender.viewer` are enough for the read-only tools; only budget provisioning needs `costsManager`/`billing.admin`. Don't run FinOps tooling as Owner.

Keep optimization a governed loop, not a one-off

• **Rightsizing/idle first, commitments second.** Act on idle and oversized resources (reversible) before buying commitments (not reversible for the term).

• **CUD purchases are money movement — human-approved, never automated.** The analyzer *identifies* commitment opportunities and their estimated savings; the purchase decision belongs to finance/FinOps under change control. Model the commitment against a stable baseline, not a spike.

• **Re-run the savings analysis on a cadence.** New workloads create new waste; a monthly savings review keeps the `FinOps – Savings Backlog – v1` current and owned.

Run FinOps as an operating rhythm

- **Monthly showback, monthly savings review, alert triage as it fires.** Put these on the named FinOps owner's calendar. FinOps decays without a rhythm.
- **Reconcile periodically.** Spot-check that the dashboard/showback totals reconcile to the console cost report for the same period, accounting for credit and export timing.
- **Extend the same discipline to Workspace and Gemini licensing.** Once GCP spend is attributed by owner, bring subscription/licensing cost into the same showback so total Google spend — not just infrastructure — has an owner.

Respect the boundaries

- **Defer financial and contractual decisions to finance/FinOps.** You provide the technical method and the evidence (attribution, budgets, savings); chargeback policy and commitment purchases are business decisions.
- **Flag preview/version-dependent surfaces.** Recommender IDs, export schema fields, and budget API fields evolve — verify them against current Google Cloud documentation for your organization rather than assuming this document's exact strings persist.

Deliverables Summary

This solution delivers a complete, self-guided FinOps package for Google Cloud: the documentation to stand the model up and run it, plus five Python/`gcloud` tools that automate discovery, provisioning, reporting, guardrail validation, and savings analysis — all safe-by-default (read-only or `--dry-run` unless `--apply` is passed), with no credentials or keys in code.

Documentation

- **Solution Overview** — the FinOps model (billing export, attribution by hierarchy/label, budgets/alerts, CUD/rightsizing optimization, showback/chargeback), the reference design, benefits, limitations, and the Assess → Design → Configure → Pilot → Validate → Expand deployment model.
- **Requirements Guide** — the FinOps-specific prerequisites (billing-admin access, a dedicated export project + BigQuery dataset, a labeling convention with measurable coverage, the Recommender API, a budget notification channel, a pilot scope and FinOps owner), each with how to verify and implement, building on the shared GCP global requirements.
- **Implementation Guide** — exact, ordered steps to enable billing export to BigQuery, build attribution views, provision budgets/alerts (pilot-first), stand up the showback dashboard, identify savings, and expand org-wide — with named artifacts, pass/fail gates, and a rollback.
- **Testing & Validation** — nine procedures (T1–T9), each with an explicit PASS/FAIL signal.

• **Troubleshooting** — common failure modes (export not appearing, view/total mismatches, alerts not firing, recommender permission errors) and their resolutions.

• **Best Practices** — durable practices for keeping attribution accurate, budgets useful, tooling safe, optimization governed, and FinOps running as an operating rhythm.

• **Executive Summary** — the outcome, approach, and why it is safe and governed.

Python tools (5)

All tools require Python 3.10+ and authenticate via `gcloud` / Application Default Credentials (`gcloud auth application-default login`). No service-account keys are created, downloaded, or committed. Where a Cloud Client Library is used, the file notes the `pip install` package.

1. `finops-discovery.py` — *Assessment / discovery (read-only).* Inventories the billing account, its linked projects, whether detailed billing export is configured, existing budgets and their thresholds, the resource hierarchy (org → folders → projects), and the labeling keys present. Produces the Assess baseline. Never changes anything.

1. `finops-budget-provisioner.py` — *Configuration / provisioning helper (dry-run by default).* Creates (or deletes) Cloud Billing budgets with alert threshold rules and a Pub/Sub/email notification target, scoped to projects, a folder, a label, or all projects. Prints the plan and changes nothing unless `--apply` is passed; `--delete` removes only the named budget.

1. `finops-report.py` — *Validation / report generator.* Emits and (with `--apply`) creates the BigQuery attribution views (`FinOps – Attribution Views – v1`) — cost by project, label, and service — and reports the **unattributed-spend %** with the top unattributed projects/services for showback and remediation.

1. `finops-guardrail-check.py` — *Policy / guardrail check (read-only).* Validates the FinOps posture against the gates: export flowing (recent rows), budgets present with expected thresholds and a live notification target, attribution views resolving, and unattributed spend below target. Exits non-zero if any check fails, for use in CI or a scheduled compliance run.

1. `finops-scope-helper.py` — *Scope / config helper.* Resolves and validates the FinOps scope —

the billing account, the export project/dataset, the pilot vs. org project sets from the resource hierarchy, and the required label keys — and writes a normalized `finops-scope.json` the other tools can consume, so the scope is defined once and reused consistently. Read-only.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

finops-budget-provisioner.py

finops-discovery.py

finops-guardrail-check.py

finops-report.py

finops-savings-analyzer.py

finops-scope-helper.py